

Apache Airflow 과제

주식 시장 데이터 파이프라인 구축 및 운영 최적화

과제 링크:

https://github.com/pjhickel/YBIGTA_DE_airflow_hw

사전 준비 사항 & 설치 가이드

강력 권장 사항 (Docker Compose): 이 과제는 Docker Compose 사용을 강력히 권장합니다! `docker compose up -d` 하나면 PostgreSQL, Airflow 웹서버, 스케줄러, 패키지 설치, Pool 생성까지 모두 자동으로 완료됩니다. 브라우저에서 `http://localhost:5050` 접속 후 바로 과제를 시작하세요.

1. Docker 환경 (추천)

Docker Desktop을 설치하고 실행한 뒤, 프로젝트 루트에서 아래 명령어를 실행합니다.

```
docker compose up -d
# http://localhost:5050 (admin / admin)
```

Docker Compose가 자동 수행: PostgreSQL 기동, DB 마이그레이션 및 admin 생성, yfinance_pool(slots=2) 생성, yfinance/pandas 설치, Webserver(5050) 및 Scheduler 기동

2. Windows 로컬 테스트 (Docker 없이)

```
pip install "apache-airflow==2.10.0" yfinance pandas
python e2e_test.py # E2E 테스트
python web_dashboard.py # 웹 대시보드 (http://localhost:5050)
```

시나리오

당신은 데이터 엔지니어입니다. 팀에서 다음과 같은 요청이 들어왔습니다.

요청 사항: 매일 주식 장 마감 후 코스피 대표 종목 5개의 주가 데이터를 자동으로 수집해주세요. 수집한 데이터를 DB에 저장하고, 이상 급등락이 발생하면 알림을 남겨주세요. 서버가 다운돼서 며칠치 데이터가 누락되더라도 소급 처리가 가능해야 합니다.

단순해 보이지만 직접 만들다 보면 Airflow의 핵심 기능들이 하나씩 필요해집니다. 이 과제는 그 필요성을 직접 체감하는 것이 목표입니다.

기본 규칙

코드를 작성하거나 설정을 변경할 때는 오직 `solution/dags/` 디렉터리 내부에서만 작업해야 합니다.

`docker-compose.yaml`, `e2e_test.py`, `web_dashboard.py`는 수정하지 마세요.

Docker 환경에서는 `solution/dags/`가 `/opt/airflow/dags/`로, `data/`는 `/opt/airflow/data/`로 마운트됩니다.

PHASE 1: 첫 번째 DAG -- 뼈대 만들기

1. 개념 이해: 논리 시간

논리 시간 (`data_interval_start`): `schedule="0 17 * * 1-5"` DAG가 2025-01-15(수) 오후 5시에 실행될 때 `context["ds"] = "2025-01-14"` (전날 날짜)입니다. 날짜를 절대 하드코딩하지 말고 반드시 `context["ds"]`를 사용하세요.

2. stock_pipeline_v1.py 작성 (solution/dags/)

DAG 설정: `dag_id="stock_pipeline_v1"`, `schedule="0 17 * * 1-5"`, `start_date=datetime.now()-timedelta(days=10)`, `catchup=True`, `default_args`에 `retries=2`, `retry_delay=timedelta(minutes=3)`

- `extract_prices`: `context["ds"]`로 yfinance 호출, 5종목 OHLCV 수집, CSV 저장
- `validate_file`: CSV 존재 확인, 행 수 == 5 검증
- `log_summary`: 처리 날짜, 종목 수, 총 행 수 출력
- Task 의존성: `extract_prices >> validate_file >> log_summary`

3. 실행 및 확인

```
docker compose exec airflow-scheduler airflow dags trigger stock_pipeline_v1
# UI: http://localhost:5050 -> Grid View -> catchup Run 자동 생성 확인
```

PHASE 2: ETL 파이프라인 -- 데이터 저장과 검증

1. 핵심 개념

TaskFlow API: @task 데코레이터로 PythonOperator를 대체합니다.

XCom: @task 함수의 return 값이 자동 저장되고, 다음 @task 인자로 자동 전달됩니다. 항상 파일 경로(str), 집계 숫자 같은 작은 메타데이터만 전달하세요.

FileSensor: 특정 파일/디렉토리 존재를 감시하는 Task. mode="reschedule"로 슬롯 반납, soft_fail=True로 timeout 시 skip 처리.

2. stock_pipeline_v2.py 작성 (solution/dags/)

모든 Task를 @task 데코레이터로 작성:

- extract_prices: CSV 파일 경로(str)를 return (빈 문자열이면 skip)
- wait_for_csv: FileSensor, mode="reschedule", poke_interval=10, timeout=120, soft_fail=True
- validate_data(csv_path): 행 수, 이상값(change_pct>30%), 결측 검증 -> dict return
- load_to_db(csv_path, quality): SQLite에 INSERT OR REPLACE (멥등성 보장)
- report_summary(quality): 처리 결과 출력
- 의존성: extract_prices >> wait_for_csv >> validate_data, 이후 XCom 의존성으로 연결

3. 멥등성 확인

같은 DAG를 2회 trigger 후 DB row 수 변화 없어야 함

```
docker compose exec airflow-scheduler airflow dags trigger stock_pipeline_v2
```

```
docker compose exec airflow-scheduler airflow dags trigger stock_pipeline_v2
```

PHASE 3: 병렬 처리와 데이터 품질 분기

1. 핵심 개념

Dynamic Task Mapping: fetch_one_ticker.expand(ticker=TICKERS)로 5개 Task 자동 생성

Pool: yfinance 429 오류 방지 -- yfinance_pool(slots=2)로 동시 호출 제한

Branching: @task.branch는 조건에 따라 다음 실행할 task_id를 반환, 나머지는 skip

Trigger Rule: notify_result에 NONE_FAILED_MIN_ONE_SUCCESS 필수 (skip된 upstream 대응)

2. 이상값 판정 기준

anomaly_rate	처리 경로	설명
rate == 0	clean_load	이상값 없음 - 그대로 DB 저장
0 < rate <= 0.20	filtered_load	이상값 행 제거 후 DB 저장
rate > 0.20	quarantine	data/quarantine/에 복사, DB 저장 안 함
전체 skip	skip_day	휴장일 - 처리 건너뛴

3. stock_pipeline_v3.py 작성 (solution/dags/)

- fetch_one_ticker: pool="yfinance_pool", 종목별 개별 수집, anomaly 판정(abs(change_pct)>30)
- aggregate: 결과 취합 + CSV 저장 + anomaly_rate 계산
- decide_path: @task.branch로 clean_load / filtered_load / quarantine / skip_day 분기
- quarantine에서는 shutil.copy2로 원본 CSV 보존하며 격리 디렉토리에 복사
- notify_result: trigger_rule=TriggerRule.NONE_FAILED_MIN_ONE_SUCCESS

4. DAG 흐름도

```
fetch[005930.KS] --+
fetch[000660.KS] -|
fetch[035420.KS] --+--> aggregate --> decide_path
fetch[005380.KS] -|
fetch[051910.KS] --+
                    +-----+-----+-----+-----+
                    v         v         v         v
              clean_load filtered_load quarantine skip
                    +-----+-----+
                    v
              notify_result
```

PHASE 4: 프로덕션 수준 완성 -- 알림과 소급 처리

1. 콜백 함수 추가 (Phase 3 코드에 추가)

- on_load_failure(context): Task 실패 시 data/alerts/{ds}_failure.txt에 장애 리포트 자동 생성
- on_pipeline_success(context): DAG 성공 시 [SUCCESS] 로그 출력
- clean_load, filtered_load에 on_failure_callback=on_load_failure 추가
- DAG 레벨에 on_success_callback=on_pipeline_success 추가

참고: clean_load와 filtered_load에는 pool 파라미터를 넣지 않습니다. 이 task들은 DB 적재만 수행하므로 yfinance API를 호출하지 않기 때문입니다.

2. Backfill (소급 처리)

서버 장애로 특정 기간의 데이터가 누락됐을 때, 그 기간을 소급해서 처리합니다.

```
# 5 영업일 소급 처리
docker compose exec airflow-scheduler airflow dags backfill \
    --start-date 2025-01-06 --end-date 2025-01-10 stock_pipeline_v4

# DB 확인 -- 각 영업일별 5행씩 적재
docker compose exec airflow-scheduler bash -c \
    "python3 -c \"import sqlite3; c=sqlite3.connect('/opt/airflow/data/stock.db');\
    print(c.execute('SELECT date, COUNT(*) FROM daily_prices GROUP BY date').fetchall())\""
```

먹등성: Backfill이 올바르게 동작하려면 반드시 먹등성이 보장되어야 합니다. 같은 기간을 재Backfill해도 DB row 수가 변하지 않아야 합니다. (INSERT OR REPLACE)

PHASE 5: 최종 확인 및 제출

1. 자동 채점 스크립트 실행

```
python e2e_test.py
```

E2E 테스트가 수행하는 검증:

- Phase 1: yfinance 데이터 수신, 5개 종목 추출, CSV 생성, 행 수/컬럼 검증
- Phase 2: SQLite 적재, 멍등성 (재실행 후 5행 유지)
- Phase 3: 종목별 fetch, 분기 결정(clean/filtered/quarantine), 경로별 처리
- Phase 4: 장애 리포트 생성, 5영업일 Backfill 시뮬레이션, 멍등성 최종 확인

2. 웹 대시보드 (선택)

```
python web_dashboard.py # http://localhost:5050
```

기능: 개별 Phase 실행, Backfill UI, DB 초기화, 실행 이력, DB 통계 차트

3. 제출 방법

solution/dags/ 디렉터리 안의 4개 파일을 제출합니다:

stock_pipeline_v1.py / stock_pipeline_v2.py / stock_pipeline_v3.py / stock_pipeline_v4.py

검증 체크리스트 (Verification Check-list)

수행 항목 (Action Taken)	예상 결과 (Expected Result)
catchup=True DAG 등록	Grid View에서 과거 날짜 Run 자동 생성
mode="reschedule" FileSensor	Task들이 슬롯 반납하며 순차 success
soft_fail=True 적용	Sensor timeout 시 실패 대신 skip 처리
동일 날짜 2회 실행 (멍등성)	DB row 수 변화 없음 (INSERT OR REPLACE)
trigger_rule 없이 Branching	notify_result가 skipped 상태
NONE_FAILED_MIN_ONE_SUCCESS	notify_result가 정상적으로 실행됨
Pool 없이 Dynamic Mapping	yfinance 429 오류 발생 가능
yfinance_pool (slots=2) 적용	fetch Task가 최대 2개씩 실행
load Task 의도적 실패 유도	data/alerts/{ds}_failure.txt 자동 생성
on_success_callback 확인	[SUCCESS] stock_pipeline_v4 완료 로그
5 영업일 Backfill 실행	DB에 날짜별 5행씩 적재
동일 기간 재Backfill	DB row 수 변화 없음 (멍등성 최종 확인)
python e2e_test.py 실행	모든 Phase PASS

yfinance 관련 주의사항

- yfinance는 Yahoo Finance의 비공식 API를 사용합니다.
- 간헐적 타임아웃은 retries=2 설정이 자동으로 재시도해줍니다.
- 일봉(1d) 데이터는 수 년치까지 무료로 가져올 수 있습니다.
- start_date를 너무 오래 전으로 설정하면 데이터가 없거나 형식이 다를 수 있으므로 최근 60일 이내를 권장합니다.